

Complejidad Asintótica e Introducción a C++



Complejidad en programación competitiva

En programación competitiva hay límites de tiempo estrictos

Un programa que exceda el límite dará el veredicto **TLE** (Time Limit Exceeded)

Necesitamos estimar que tan rápido será nuestro código antes de enviarlo



¿Cómo puedo calcular cuánto se demora mi programa?

Medir con un cronómetro depende del entorno (hardware, lenguaje, etc.)

Buscamos un modelo teórico que anticipe el rendimiento

Hoy aprenderemos a estimar ese tiempo de forma práctica



Antes de comenzar: ¿Qué es un algoritmo?

¿Qué es un algoritmo?



Antes de comenzar: ¿Qué es un algoritmo?

Secuencia finita de pasos bien definidos

Transforman una entrada en una salida

Deben ser correctos y, para competir, eficientes



¿Cómo sé cuál algoritmo es más rápido?

Supongamos dos algoritmos que resuelven el mismo problema

¿Cuál elijo?



¿Cómo sé cuál algoritmo es más rápido?

Factores externos que influyen:

- Lenguaje de programación (C++, Python, Java...)
- Potencia del computador
- Sistema operativo y compilador

Necesitamos una herramienta que abstraiga estos factores.

Queremos encontrar funciones:
que indiquen el tiempo según el tamaño de entrada .
Luego, simplemente comparamos y .

El problema práctico

Calcular exactamente es casi imposible

Depende de constantes ocultas y del hardware



La solución

Renunciamos a saber la constante exacta

Nos enfocamos en cómo crece cuando aumenta



Crecimiento de funciones

Digamos por ejemplo que y



Crecimiento de funciones

A medida que crece, domina a

Para suficientemente grande, es mucho mayor



Notación asintótica O

Describe cómo crece

Ignora constantes y detalles de hardware

$O(f)$ es el conjunto de funciones que crecen como máximo tan rápido como f .

¿Cómo calcular la notación O?

Estimaremos la cantidad de operaciones que realiza un algoritmo, dependiendo de la entrada .

Usaremos la notación O para expresar el crecimiento de estas operaciones.

¿Cómo calcular la notación O?

Ejemplo N°1

```
def es_primo(x):  
    for i in range(2, x):  
        if x % i == 0:  
            return False  
    return True  
  
n = int(input())  
  
contador = 0  
for i in range(2, n + 1):  
    if es_primo(i):  
        contador += 1  
  
print(contador)
```

El algoritmo cuenta los números primos en el rango

¿Cuántas operaciones hace?

¿Cómo calcular la notación O?

Ejemplo N°2

```
def es_primo(x):  
    for i in range(2, x):  
        if x % i == 0:  
            return False  
    return True  
  
n = int(input())  
  
contador = 1  
for i in range(3, n + 1, 2):  
    if es_primo(i):  
        contador += 1  
  
print(contador)
```

Optimización del programa anterior — ahora no revisa los números pares

- El bucle externo itera veces

¿Cómo calcular la notación O?

Ejemplo N°3

```
def es_primo(x):  
    for i in range(2, int(x**0.5) + 1):  
        if x % i == 0:  
            return False  
    return True  
  
n = int(input())  
  
contador = 0  
for i in range(2, n + 1):  
    if es_primo(i):  
        contador += 1  
  
print(contador)
```

Otra optimización — ahora `es_primo(x)` solo itera hasta

- El bucle externo itera veces

¿Cómo calcular la notación O?

Criba de Eratóstenes

```
n = int(input())

es_primo = [True] * (n + 1)
es_primo[0] = es_primo[1] = False

for i in range(2, n + 1):
    if es_primo[i]:
        for j in range(i * 2, n + 1, i):
            es_primo[j] = False

contador = 0
for i in range(2, n + 1):
    if es_primo[i]:
        contador += 1
print(contador)
```

- El bucle externo itera veces
- El bucle interno itera veces en el peor de los casos



¿Cómo estimo el tiempo de ejecución?

Una vez que tenemos la complejidad, podemos estimar el tiempo de ejecución

En programación competitiva, se suele usar el estimado de operaciones por segundo

Para un algoritmo con complejidad $O(n^2)$, el tiempo de ejecución se estima como:



¿Cómo estimo el tiempo de ejecución?

Por ejemplo, si $y = 10^4$, entonces:

operaciones

El programa debería ejecutarse en aproximadamente 10.000 segundos (~2.8 horas)

Por otro lado, si $y = 10$, entonces:

operaciones

El programa debería ejecutarse en aproximadamente 0.2 segundos



Introducción a C++



¿Por qué C++ en programación competitiva?

- Lenguaje de bajo nivel
- Gran velocidad, comparable al C "puro"
- Biblioteca estándar muy completa (STL)
- Es el lenguaje dominante en la mayoría de los jueces en línea



¿Cómo instalar C++?

Sigue la guía recomendada para instalar el compilador de C++ en el editor de tu preferencia:

Compilación y editores — Apunte ProgComp UChile

Instrucciones para Linux, Windows y Mac

Recomendaciones de editores y entornos

Ejemplos de comandos para compilar y ejecutar tus programas



Plantilla mínima

```
#include <bits/stdc++.h>    // Toda la STL en un solo include
using namespace std;

int main() {
    /* - tu solución - */
    return 0;
}
```

`bits/stdc++.h` no es parte del estándar, pero todos los jueces que usan GCC lo incluyen.



Tipos de variables básicos

Tipo	Rango aproximado / uso
<code>int</code>	$\pm 2 \cdot 10^9$
<code>long long</code>	$\pm 9 \cdot 10^{18}$
<code>double</code>	15 dígitos decimales
<code>char</code>	1 byte
<code>string</code>	Cadena de caracteres dinámicos
<code>vector<T></code>	Arreglo dinámico

Usa siempre `long long` cuando necesites enteros mayores a $\pm 2 \cdot 10^9$.



Entrada / salida

```
int a;  
int b;  
  
cin >> a;  
cin >> b;  
  
cout << a + b;  
cout << endl;
```

Input

3 7

Output

10



Entrada / salida

```
int a, b;  
  
cin >> a >> b;  
  
cout << a + b << endl;
```

Input

3 7

Output

10



Condicionales

```
int a;  
cin >> a;  
  
if (a > 0) {  
    cout << "Positivo" << endl;  
} else if (a < 0) {  
    cout << "Negativo" << endl;  
} else {  
    cout << "Cero" << endl;  
}
```



Tipos principales:

- `for` : Repetición con contador
- `while` : Repetición mientras se cumpla una condición

```
for (int i = 0; i < n; i++) {  
    cout << i << " ";  
}  
// Output: 0 1 2 3 4
```



```
int j = 0;
while (j < n) {
    cout << j << " ";
    j++;
}
// Output: 0 1 2 3 4
```



Funciones

```
int suma(int a, int b) {  
    return a + b;  
}
```

- El tipo va antes del nombre
- Los argumentos se pasan por valor (copia)
- Ejemplo: `cout << suma(3, 7) << endl;` muestra `10`



```
vector<int> v;           // vacío

v.push_back(42);        // añade al final
v.push_back(7);         // añade al final
v.push_back(13);        // añade al final

cout << v.size() << endl; // muestra 3

cout << v[1] << endl; // muestra 7
v.pop_back();           // elimina el último elemento

cout << v.back() << endl; // muestra 7
```

Capacidad automática, memoria contigua.

Métodos útiles: `push_back()`, `size()`, `clear()`, `back()`, `pop_back()`.



vector

```
int n;  
cin >> n;  
vector<int> v(n);  
  
for (int i = 0; i < n; i++) {  
    cin >> v[i];  
}  
  
v[1] = 42;  
  
for (int i = 0; i < n; i++) {  
    cout << v[i] << " ";  
}
```

Input

5

9 4 8 1 3



Resolviendo problemas en C++

- Problema "Watermelon" - Codeforces
- Problema "Vanya and Fence" - Codeforces